LINQ & Entity Framework

WITH ELWAKEEL

From Concepts to Clarity

PRE-SESSION INTELLIGENCE PACK



Prepare Handout

LINQ and EF - Session03 - Abdelrahman Elwakeel

أول حاجة: الانتقال من المجموعات الاستاتيكية لقواعد البيانات

إيه السياق العام؟
في المحاضرات اللي فاتت، اتعاملنا مع برمجيات LINQ على مستوى المجموعات الاستاتيكية (Static Collections).

طيب إيه الهدف الفعلي؟
إحنا مش بنتعلم LINQ عشان نكتبها ضد ملفات XML أو مجموعات استاتيكية بس. الهدف الأساسي إننا نكتبها ضد أنظمة إدارة قواعد البيانات (Database Management Systems) زي SQL Server أو Oracle أو Mongo.

المهمة الحالية إيه؟
نبدأ في عملية الربط (Mapping) بين طبقة قاعدة البيانات (Database Layer) وتطبيق السي شارب (C# Application).

تاني حاجة: تقنيات الربط (Mapping Technologies)

إيه التقنيات الأساسية للمابينج؟
في 3 مصطلحات أساسية لازم المبرمج يكون عارفهم:

ADO.NET:
دي الأساس والطبقة السفلى اللي بتشتغل عليها باقي التقنيات.

Dapper:
بيتصنف كـ "Micro ORM". بيجمع بين سرعة ADO.NET (استخدام SQL صريح) وبين قدرة الـ Mapping على عمل EF للبيانات بشكل بسيط.

Entity Framework (EF):
ADO.NET متكامل، بيشتغل كطبقة فوق (Object-Relational Mapper) ده "Full ORM" عشان يخفي تفاصيل التنفيذ المباشرة ويوفر ميزات برمجية عالية المستوى.

💡 نصيحة للمبتدئين:
ممكن ما يُطلبش منك تشتغل بـ ADO.NET أو Dapper في البداية، بس مايُغتفرش لك إنك ما تعرفش وجودهم. لازم تعرف إن EF هو وسيط بيخفي تعقيدات ADO.NET عشان يوفرلك ميزات إضافية.

تالت حاجة: مناهج العمل في Entity Framework

إيه النهجين اللي بيوفرهم الـ EF؟

قاعدة البيانات أولاً (Database First):

المبدأ:
بتبدأ من بناء الجداول والحاجات جوا قاعدة البيانات الأول.

العملية:
الـ ORM بيحول الجداول (Tables) لكلاسات (Classes) ومجموعات (Collections) في السي شارب.

الكود أولاً (Code First):

المبدأ:
بتبدأ بكتابة الكلاسات والمجموعات في السي شارب الأول.

العملية:
الـ ORM بيترجم الكود ده لحاجة تمثله في قاعدة البيانات (Tables).

رابع حاجة: ليه بنفضل منهج Code First؟

في الكورس ده بنركز على Code First ليه؟
بنركز عليه بنسبة تتخطى 95% لـ 4 أسباب جوهرية:

تحقيق حلم "السي شارب فقط":
بتقدر تكتب منطق النظام كله بلغة السي شارب بس، وهي اللي هتدير نظام قواعد البيانات بالتبعية.

التصميم الموجه للكائنات (Object-Oriented Design):
بدل ما ترسم (ERD) معقد، هتعمل (Class Diagram) بيمثل الواقع. تقدر تستغل مبادئ الـ OOP زي الوراثة (Inheritance)، التجميع (Aggregation)، والارتباط (Association) عبر حاجة اسمها **Navigation Properties**.

دعم الأنظمة المتعددة (Multi-DBMS Support):
بتكتب الاستراكشر مرة واحدة بالسي شارب، وممكن تترجمه لأي نظام (SQL Server, Oracle, MySQL) بمجرد تغيير "المزود" (Provider).

الصيانة والنسخ (Maintainability & Versioning):
بيوفر ميزة **المايجريشن (Migrations)** اللي بتتبع التغييرات في الهيكل (Schema) بدقة، وده بيسمحلك تعمل تراجع (Rollback) أو تتبع تطور المشروع زمنياً.

خامس حاجة: تعريف كلاسات الكيانات (Entity Classes)

إيه أول خطوة في أي مشروع؟
تعريف الكلاسات اللي بتمثل البزنس، وليها 3 مسميات (Models, Entities, POCO)

.(Classes

ممكن مثال على كلاس الموظف؟
بيتحط الكلاسات دي جوا فولدر اسمه **Models** للتنظيم.

```
public class Employee
{
    // هيبقى Primary Key تلقائياً (By Convention)
    public int Id { get; set; }

    // String بيقبل Null افتراضياً (Optional)
    public string? Name { get; set; }

    // Value Type ما بيقبلش Null (Required)
    public decimal Salary { get; set; }

    // Nullable int (Optional)
    public int? Age { get; set; }
}
```

يعني إيه POCO؟
اختصار لـ Plain Old CLR Object، ودي كلاسات فيها خصائص (Properties) بس عشان تمثل البيانات.

سادس حاجة: سياق قاعدة البيانات (DBContext)

يعني إيه DBContext؟
ده الكلاس المسؤول عن التواصل مع قاعدة البيانات (Database Context). يعني السياق اللي بيتم فيه التواصل.

إيه قواعد تصميم الـ DBContext؟

التسمية:
لازم اسم الكلاس ينتهي بكلمة `DbContext` (زي `CompanyDbContext`) عشان يتعرف عليه فوراً.

الموقع:
بيتحط في فولدر مستقل اسمه **Contexts** أو **DbContexts** لأن المشروع ممكن يكون فيه أكتر من قاعدة بيانات.

الوظيفة:
بيحدد السيرفر، طريقة الدخول (Authentication)، واسم قاعدة البيانات.

سابع حاجة: التعامل مع الحزم (NuGet Packages)

ليه محتاجين نحمل حزم؟
بما إن دوت نت بقى "Component-Based"، مكتبات EF مابتنزلش تلقائياً مع المشروع، لازم تحملها يدوياً باستخدام **NuGet Package Manager**.

إيه الحزم المطلوبة؟

`Microsoft.EntityFrameworkCore.SqlServer`:
كمزود بيانات SQL Server للتعامل مع.

`Microsoft.EntityFrameworkCore.Tools`:
في الكونسول (Migrations) لتفعيل أوامر المايجريشن.

⚠️ تحذير مهم عن الإصدارات:
لازم تحمل إصدار متوافق مع نسخة الدوت نت اللي بتشتغل بيها. لو بتشتغل بـ .NET 5، حمل نسخة x.5 من الحزم.

تامن حاجة: برمجة كلاس الـ Context

إزاي بنبرمج كلاس الـ Context؟
لازم الكلاس يرث من `DbContext` ونعمل **Override** لدالة `OnConfiguring` عشان نبعت بيانات الاتصال (Connection String).

```
public class CompanyDbContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsB
uilder optionsBuilder)
    {
        // تحديد المزود (SQL Server) وبيانات الاتصال
        optionsBuilder.UseSqlServer("Server=.;Database=Comp
anyDB;Trusted_Connection=True;");
    }

    // تمثيل الجدول في قاعدة البيانات
    public DbSet<Employee> Employees { get; set; }
}
```

يعني إيه DbSet؟
بتمثل "المجموعة" اللي هتتحول لجدول في قاعدة البيانات.

تاسع حاجة: ميكانيكا المايجريشن (Migrations)

يعني إيه Migration؟
المايجريشن هي الوسيلة الذكية لنقل التغييرات من السي شارب لقاعدة البيانات من غير ما نهدم البيانات القديمة.

إيه مكونات ملف المايجريشن؟
لما تكتب أمر `Add-Migration InitialCreate` في الكونسول، بيتعمل فولدر فيه:

Snapshot:
صورة لآخر وضع كانت عليه قاعدة البيانات للمقارنة مستقبلاً.

Up Method:
فيها الكود اللي هينفذ لما "تطبق" التغييرات (زي إنشاء جدول).

Down Method:
فيها الكود اللي هينفذ لما "تتراجع" عن التغييرات (زي حذف الجدول).

ليه بنستخدم التوقيت (Timestamp) في أسماء الملفات؟
بيتضاف أرقام بتمثل السنة والشهر واليوم والثانية عشان نضمن:

اليونيك (Uniqueness):
إن اسم الملف يكون فريد دايماً.

الترتيب (Ordering):
الحفاظ على تسلسل تنفيذ التغييرات من الأقدم للأحدث.

عاشر حاجة: طرق الربط (Mapping Strategies)

في كام طريقة للتحكم في التحويل؟
في 3 طرق للتحكم في إزاي الكلاس يتحول لجدول:

الربط بالاتفاق (Mapping by Convention):
ده السلوك الافتراضي (Default Behavior).
مثال: أي خاصية اسمها `Id` بتبقى Primary Key تلقائياً وبتاخد خاصية (1,1) Identity.

وسوم البيانات (Data Annotations):
استخدام "Attributes" فوق الخصائص مباشرة.
مثال: `[Key]` لتحديد المفتاح، `[Required]` لمنع النل، `[Table("Employees")]` لتغيير اسم الجدول.

الـ Fluent API (الأقوى والأفضل):
بتتم عبر كود جوا دالة `OnModelCreating`.

ليه دي الأفضل؟

1. بتسمح بإعدادات متقدمة (Advanced Configuration) مش موجودة في الوسوم.

2. بتحافظ على "نظافة الموديل" (Clean POCO) حيث الموديل بيفضل خالي من وسوم قواعد البيانات.

3. ليها **الأولوية القصوى** في التنفيذ فوق باقي الطرق.

حداشر حاجة: المستويات المتقدمة في Fluent API

إيه المستويات المتقدمة؟

المستوى الأول: الكتابة المباشرة
كتابة الأوامر ورا بعض جوا `OnModelCreating`.

المستوى الثاني: استخدام Lambda Expressions
عشان تقلل تكرار اسم الكلاس وتخلي الكود أكثر سلاسة.

المستوى الثالث: كلاسات الإعدادات المنفصلة (IEntityTypeConfiguration)
بيتعمل كلاس منفصل لكل كيان (زي `EmployeeConfiguration`).

إيه الميزة؟
تنظيم هائل للكود، حيث كلاس الـ Context مابيتضخمش بآلاف الأسطر البرمجية.

إزاي بيستخدم؟
بيتستدعى في الـ Context باستخدام `modelBuilder.ApplyConfiguration(new EmployeeConfiguration());`.

اتناشر حاجة: أوامر الكونسول الأساسية

إيه الأوامر الأساسية؟

`Add-Migration [Name]`:
لتجهيز التغييرات (Prepare Changes).

`Update-Database`:
لتطبيق التغييرات فعلياً على قاعدة البيانات (Apply Changes).

`Remove-Migration`:
لحذف آخر مايجريشن **بشرط** ألا يكون اتطبق على قاعدة البيانات.

⚠️ **قاعدة ذهبية:**
ما تعدلش أبداً كود الـ `Up` يدوياً أو تعدل الهيكل في قاعدة البيانات يدوياً. دايماً خلي الكود هو المصدر الوحيد للحقيقة (Source of Truth) عبر المايجريشن.
